

```
best_split = splitted[0]
lowest_error = splitted[1]
best_var = var
best_lowermean = splitted[2]
best_highermean = splitted[3]

generated_tree = [[best_var, float('-inf'), best_split, best_lowermean],
[best_var, best_split, \      float('inf'), best_highermean]]

return(generated_tree)
```

[Listing 9-3](#): A function that iterates over every variable and finds the best variable to split on

In [Listing 9-3](#), we’ve defined a function with a `for` loop that iterates over all the variables in a list of variables. For each of those variables, it finds the best split point by calling the `get_splitpoint()` function. Each variable, split at its best split point, will lead to a certain error sum for our predictions. If a particular variable has a lower error sum than any previous variable we considered, we’ll store that variable name as `best_var`. After looping through every variable name, it has found the variable with the lowest error sum, stored in `best_var`. We can run this code on a set of variables other than `sclmeet` as follows:

```
variables = ['rlgdgr', 'hhmb', 'netusoft', 'agea', 'eduyrs']
outcome_variable = 'happy'
print(getsplit(ess, variables, outcome_variable))
```

In this case, we see the following output:

```
[['netusoft', -inf, 4.0, 7.041597337770383], ['netusoft', 4.0, inf,
7.73042471042471]]
```

Our `getsplit()` function has output a very simple “tree” in the form of a nested list. This tree has only two branches. The first branch is represented by the first nested list, and the second branch is represented by the second nested list. Each element of both nested lists tells us something about their respective branches. The first list tells us that we’re looking at a branch based on a respondent’s value of `netusoft` (frequency of internet usage). Specifically, the first branch corresponds to people whose value of `netusoft` is between `-inf` and `4.0`, where `inf` stands for infinity. In other words, people